Exploration of temporal dynamics

Scott Forrest

2025-09-15

In this script we demonstrate different approaches to including covariates in an SSF model.

Table of contents

Load required packages

```
library(tidyverse)
packages <- c("amt", "sf", "terra", "RColorBrewer", "leaflet")
walk(packages, require, character.only = T)
```

Import data and clean

```
buffalo_data <- read_csv("data/buffalo.csv")
```

```
New names:
Rows: 133161 Columns: 11
-- Column specification
-------------------------------------------------------- Delimiter: "," chr
(2): node, dates dbl (7): ...1, lat, lon, height, accuracy, heading, speed dttm
(2): timestamp, DateTime
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `` -> `...1`
```

```
# remove individuals that have poor data quality or less than about 3 months of data.
# The "2014.GPS_COMPACT copy.csv" string is a duplicate of ID 2024, so we exclude it
buffalo_data <- buffalo_data %>% filter(!node %in% c("2014.GPS_COMPACT copy.csv",
                                          2029, 2043, 2265, 2284, 2346))

buffalo_data <- buffalo_data %>%
  group_by(node) %>%
  arrange(DateTime, .by_group = T) %>%
  distinct(DateTime, .keep_all = T) %>%
  arrange(node) %>%
  mutate(ID = node)

buffalo_clean <- buffalo_data[, c(12, 2, 4, 3)]
colnames(buffalo_clean) <- c("id", "time", "lon", "lat")
attr(buffalo_clean$time, "tzone") <- "Australia/Queensland"
head(buffalo_clean)
```

```
# A tibble: 6 x 4
  id    time                  lon   lat
  <chr> <dttm>              <dbl> <dbl>
```

```
1 2005  2018-07-25 10:04:02  134. -12.3
2 2005  2018-07-25 11:04:23  134. -12.3
3 2005  2018-07-25 12:04:39  134. -12.3
4 2005  2018-07-25 13:04:17  134. -12.3
5 2005  2018-07-25 14:04:39  134. -12.3
6 2005  2018-07-25 15:04:27  134. -12.3
```

```
tz(buffalo_clean$time)
```

```
[1] "Australia/Queensland"
```

```
buffalo_ids <- unique(buffalo_clean$id)
```

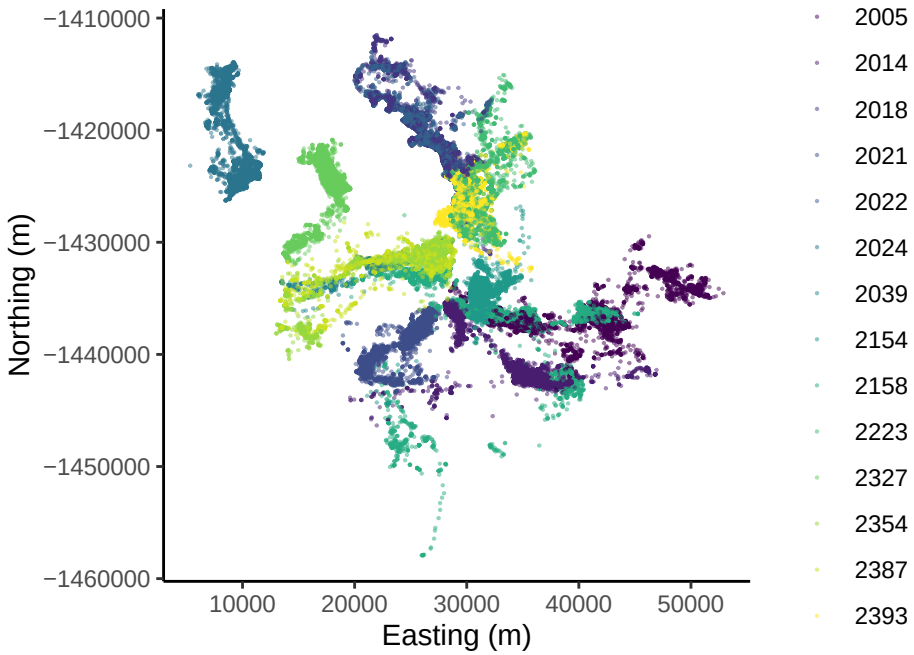
Create a step object

Use the `amt` package to create a trajectory object from the cleaned data.

```
buffalo_all <- buffalo_clean %>% mk_track(id = id,
                                          lon,
                                          lat,
                                          time,
                                          all_cols = T,
                                          crs = 4326) %>%
  transform_coords(crs_to = 3112, crs_from = 4326) # Transformation to GDA94 /
# Geoscience Australia Lambert (https://epsg.io/3112)
```

Plot the data spatially

```
buffalo_all %>%
  ggplot(aes(x = x_, y = y_, colour = id)) +
  geom_point(alpha = 0.5, size = 0.1) +
  coord_fixed() +
  scale_x_continuous("Easting (m)") +
  scale_y_continuous("Northing (m)") +
  scale_colour_viridis_d() +
  theme_classic() +
  theme(legend.position = "right")
```




```
# ggsave("outputs/data_prep/buffalo_djelk_map.png",
#        width = 150, height = 150, units = "mm",  dpi = 600)
```

Show data on a satellite basemap

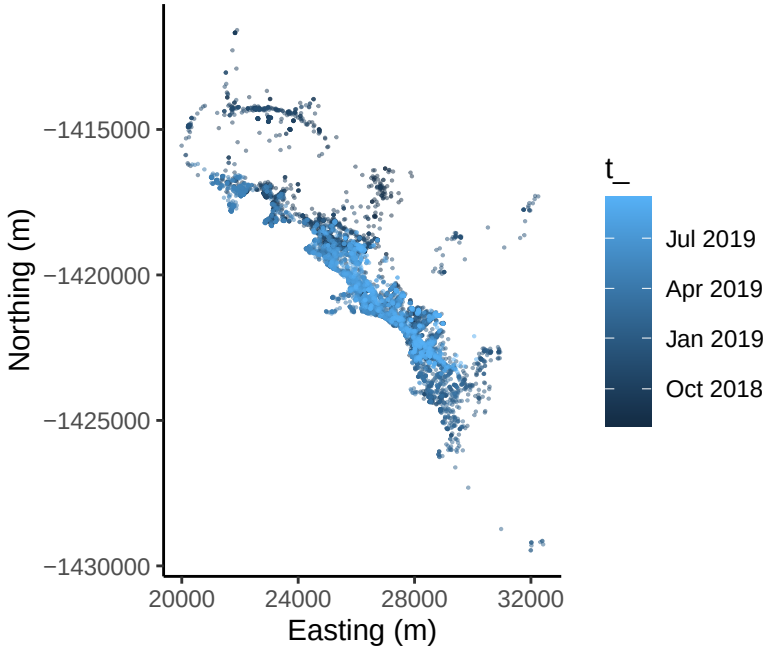
If viewing as a PDF this will be an automated screenshot. It viewing the HTML file this should be interactive.

```
# Use the original longitude/latitude data for mapping
leaflet(data = buffalo_clean) %>%
  addProviderTiles(providers$Esri.WorldImagery) %>%
  addCircleMarkers(
    lng = ~lon, lat = ~lat,
    color = "red", radius = 0.5, opacity = 0.5,
    popup = ~paste("ID:", id, "<br>", "Time:", time)
  ) %>%
  addLayersControl(
    baseGroups = c("Satellite"),
    options = layersControlOptions(collapsed = TRUE)
  )
```


+
−
Leaflet | Tiles © Esri — Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User Community

Pick out a single individual

```
which_buffalo <- "2022" # select a single buffalo ID

buffalo_id <- buffalo_all %>% filter(id == which_buffalo)

buffalo_id %>%
  ggplot(aes(x = x_, y = y_, colour = t_)) +
  geom_point(alpha = 0.5, size = 0.1) +
  coord_fixed() +
  scale_x_continuous("Easting (m)") +
  scale_y_continuous("Northing (m)") +
  theme_classic()
```



Create steps

We will just use a steps object here for now

```
buffalo_id_steps <- buffalo_id %>%
  steps()

head(buffalo_id_steps)
```

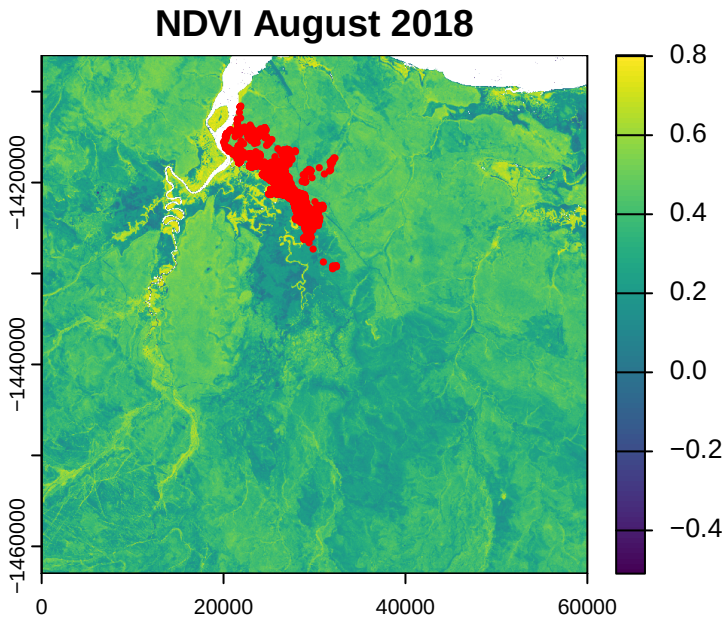
```
# A tibble: 6 x 10
     x1_    x2_       y1_       y2_   sl_ direction_p    ta_ t1_
*  <dbl>  <dbl>     <dbl>     <dbl> <dbl>       <dbl>  <dbl> <dttm>
1 27929. 27931. -1421151. -1421156.  5.95     -1.29   NA     2018-07-
25 10:35:16
2 27931. 27938. -1421156. -1421157.  7.42     -0.0187  1.27  2018-07-
25 11:36:05
3 27938. 27934. -1421157. -1421155.  3.89      2.80    2.82  2018-07-
25 12:34:52
4 27934. 27938. -1421155. -1421160.  5.98     -0.924   2.56  2018-07-
25 13:34:14
5 27938. 27938. -1421160. -1421157.  3.37      1.52    2.44  2018-07-
25 14:34:08
6 27938. 27940. -1421157. -1421154.  2.92      0.884  -0.635 2018-07-
25 16:34:33
# i 2 more variables: t2_ <dttm>, dt_ <drtn>
```

Import spatial covariates

NDVI

Although NDVI changes over time, and we have access to monthly layers, we will just select a single month here.

```
ndvi <- rast("mapping/ndvi_aug_2018.tif")
plot(ndvi, main = "NDVI August 2018")
points(buffalo_id$x_, buffalo_id$y_, col = "red", pch = 16, cex = 0.5)
```

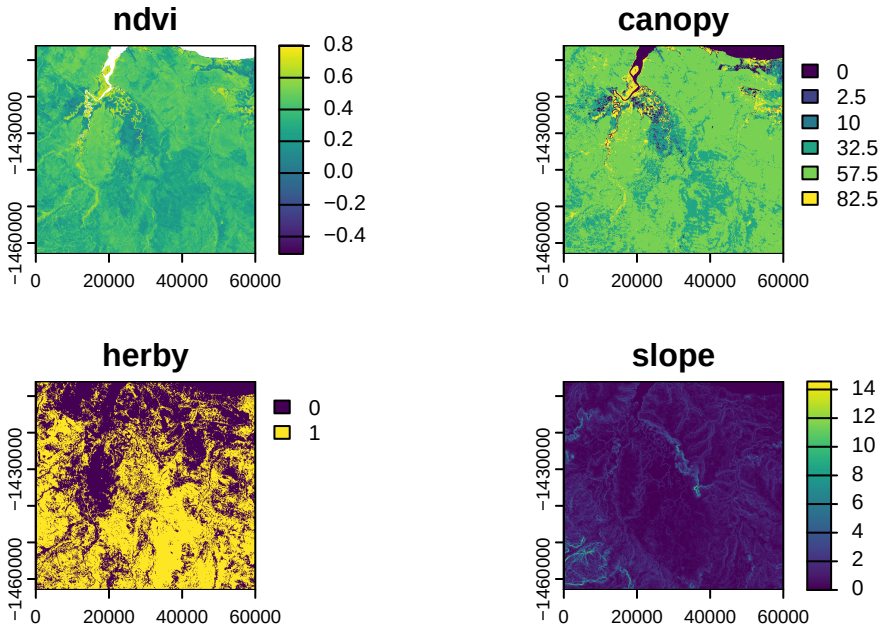



```
ndvi
```

```
class       : SpatRaster
dimensions  : 2280, 2400, 1  (nrow, ncol, nlyr)
resolution  : 25, 25  (x, y)
extent      : 0, 60000, -1463000, -1406000  (xmin, xmax, ymin, ymax)
coord. ref. : GDA94 / Geoscience Australia Lambert (EPSG:3112)
source      : ndvi_aug_2018.tif
name        :       ndvi
min value   : -0.5441047
max value   :  0.8086554
```

Other covariates

```
canopy <- rast("mapping/canopy_cover.tif")
herby <- rast("mapping/veg_herby.tif")
slope <- rast("mapping/slope_raster.tif")

spatial_covs <- c(ndvi, canopy, herby, slope)
names(spatial_covs) <- c("ndvi", "canopy", "herby", "slope")
plot(spatial_covs)
```



Extract covariate values at the end of each step

```
buffalo_id_steps <- buffalo_id_steps %>%
  extract_covariates(covariates = spatial_covs, where = "end")

head(buffalo_id_steps)
```

```
# A tibble: 6 x 14
     x1_    x2_       y1_       y2_   sl_ direction_p    ta_ t1_
*  <dbl>  <dbl>     <dbl>     <dbl> <dbl>       <dbl>  <dbl> <dttm>
1 27929. 27931. -1421151. -1421156.  5.95     -1.29   NA     2018-07-
25 10:35:16
2 27931. 27938. -1421156. -1421157.  7.42     -0.0187  1.27  2018-07-
25 11:36:05
3 27938. 27934. -1421157. -1421155.  3.89      2.80    2.82  2018-07-
25 12:34:52
4 27934. 27938. -1421155. -1421160.  5.98     -0.924   2.56  2018-07-
25 13:34:14
5 27938. 27938. -1421160. -1421157.  3.37      1.52    2.44  2018-07-
25 14:34:08
6 27938. 27940. -1421157. -1421154.  2.92      0.884  -0.635 2018-07-
25 16:34:33
# i 6 more variables: t2_ <dttm>, dt_ <drtn>, ndvi <dbl>, canopy <dbl>,
#   herby <dbl>, slope <dbl>
```

Prepare data for extracting covariate information

```
buffalo_id_steps <- buffalo_id_steps %>%
  mutate(t1_ = lubridate::with_tz(buffalo_id_steps$t1_, tzone = "Australia/Darwin"),
         t2_ = lubridate::with_tz(buffalo_id_steps$t2_, tzone = "Australia/Darwin"))

buffalo_id_steps <- buffalo_id_steps %>%
  mutate(x1 = x1_, x2 = x2_,
         y1 = y1_, y2 = y2_,
         t1 = t1_,
         t1_rounded = round_date(t1_, "hour"),
         hour_t1 = hour(t1_rounded),
         t2 = t2_,
         t2_rounded = round_date(t2_, "hour"),
         hour_t2 = hour(t2_rounded),
         hour_t2 = ifelse(hour_t2 == 0, 24, hour_t2),
         yday = yday(t1_),
         year = year(t1_),
         month = month(t1_),
         sl = sl_,
         log_sl = log(sl_),
         ta = ta_,
         cos_ta = cos(ta_),
         canopy_01 = canopy/100)

head(buffalo_id_steps)
```

```
# A tibble: 6 x 32
     x1_    x2_       y1_       y2_   sl_ direction_p    ta_ t1_
*  <dbl>  <dbl>     <dbl>     <dbl> <dbl>       <dbl>  <dbl> <dttm>
1 27929. 27931. -1421151. -1421156.  5.95     -1.29   NA     2018-07-
25 10:05:16
2 27931. 27938. -1421156. -1421157.  7.42     -0.0187  1.27  2018-07-
25 11:06:05
3 27938. 27934. -1421157. -1421155.  3.89      2.80    2.82  2018-07-
25 12:04:52
4 27934. 27938. -1421155. -1421160.  5.98     -0.924   2.56  2018-07-
25 13:04:14
5 27938. 27938. -1421160. -1421157.  3.37      1.52    2.44  2018-07-
25 14:04:08
6 27938. 27940. -1421157. -1421154.  2.92      0.884  -0.635 2018-07-
25 16:04:33
# i 24 more variables: t2_ <dttm>, dt_ <drtn>, ndvi <dbl>, canopy <dbl>,
```

```
#   herby <dbl>, slope <dbl>, x1 <dbl>, x2 <dbl>, y1 <dbl>, y2 <dbl>,
#   t1 <dttm>, t1_rounded <dttm>, hour_t1 <int>, t2 <dttm>, t2_rounded <dttm>,
#   hour_t2 <dbl>, yday <dbl>, year <dbl>, month <dbl>, sl <dbl>, log_sl <dbl>,
#   ta <dbl>, cos_ta <dbl>, canopy_01 <dbl>
```

Hourly movement behaviour and selection of covariates

Here we bin the trajectories into the hours of the day, and calculate the mean, and quantiles for the step lengths and four habitat covariates. This is a similar approach to in Forrest et al. (2024) and Forrest et al. (2025), where we used this approach to assess temporal dynamics as an exploratory step, but also as a validation step by assessing whether the simualted trajectories had also learned the temporally dynamic behaviour.

Observed buffalo data

```
buffalo_hourly_habitat <-
  buffalo_id_steps %>% dplyr::group_by(hour_t2) %>%
  summarise(n = n(),

            # step lengths
            step_length_mean = mean(sl_),
            step_length_q025 = quantile(sl_, 0.025),
            step_length_q25 = quantile(sl_, 0.25),
            step_length_median = quantile(sl_, 0.5),
            step_length_q75 = quantile(sl_, 0.75),
            step_length_q975 = quantile(sl_, 0.975),

            # ndvi
            ndvi_mean = mean(ndvi),
            ndvi_q025 = quantile(ndvi, 0.025),
            ndvi_q25 = quantile(ndvi, 0.25),
            ndvi_median = median(ndvi),
            ndvi_q75 = quantile(ndvi, 0.75),
            ndvi_q975 = quantile(ndvi, 0.975),

            # herby
            herby_mean = mean(herby),
            herby_q025 = quantile(herby, 0.025),
            herby_q25 = quantile(herby, 0.25),
            herby_median = median(herby),
            herby_q75 = quantile(herby, 0.75),
            herby_q975 = quantile(herby, 0.975),
```

```
            # canopy cover
            canopy_mean = mean(canopy_01),
            canopy_q025 = quantile(canopy_01, 0.025),
            canopy_q25 = quantile(canopy_01, 0.25),
            canopy_median = median(canopy_01),
            canopy_q75 = quantile(canopy_01, 0.75),
            canopy_q975 = quantile(canopy_01, 0.975),

            # slope
            slope_mean = mean(slope),
            slope_q025 = quantile(slope, 0.025),
            slope_q25 = quantile(slope, 0.25),
            slope_median = median(slope),
            slope_q75 = quantile(slope, 0.75),
            slope_q975 = quantile(slope, 0.975)

            ) %>% ungroup()

head(buffalo_hourly_habitat)
```

```
# A tibble: 6 x 32
  hour_t2     n step_length_mean step_length_q025 step_length_q25
    <dbl> <int>            <dbl>            <dbl>           <dbl>
1       1   394            168.             0.721            3.59
2       2   400            175.             1.08             4.78
3       3   398            160.             0.713            4.06
4       4   403            164.             0.791            3.81
5       5   405            112.             0.770            3.21
6       6   396             76.1            0.659            2.63
# i 27 more variables: step_length_median <dbl>, step_length_q75 <dbl>,
#   step_length_q975 <dbl>, ndvi_mean <dbl>, ndvi_q025 <dbl>, ndvi_q25 <dbl>,
#   ndvi_median <dbl>, ndvi_q75 <dbl>, ndvi_q975 <dbl>, herby_mean <dbl>,
#   herby_q025 <dbl>, herby_q25 <dbl>, herby_median <dbl>, herby_q75 <dbl>,
#   herby_q975 <dbl>, canopy_mean <dbl>, canopy_q025 <dbl>, canopy_q25 <dbl>,
#   canopy_median <dbl>, canopy_q75 <dbl>, canopy_q975 <dbl>, slope_mean <dbl>,
#   slope_q025 <dbl>, slope_q25 <dbl>, slope_median <dbl>, slope_q75 <dbl>, ...
```

Lengthen the dataframe

```
buffalo_hourly_habitat_long <- buffalo_hourly_habitat %>%
  pivot_longer(cols = !c(hour_t2), names_to = "variable", values_to = "value") %>%
  filter(!variable == "n")
```

```r
head(buffalo_hourly_habitat_long)
```

```
# A tibble: 6 x 3
  hour_t2 variable              value
    <dbl> <chr>                 <dbl>
1       1 step_length_mean    168.
2       1 step_length_q025      0.721
3       1 step_length_q25       3.59
4       1 step_length_median   34.8
5       1 step_length_q75     188.
6       1 step_length_q975   1028.
```

Set plotting parameters

```r
# set plotting parameters here that will change in each plot

# path
path_linewidth <- 0.5
path_alpha <- 0.1
path_95_alpha <- 1

# ribbon
ribbon_95_alpha <- 0.5
ribbon_50_alpha <- 0.25
```

Loop over each variable to create a plot

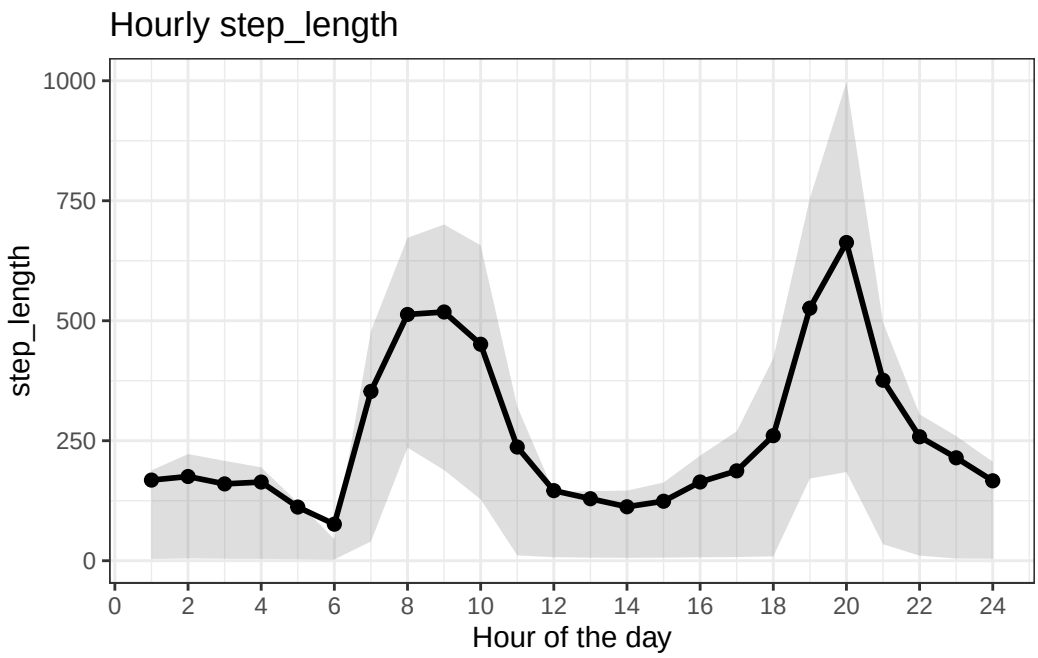
As we have some categorical and binary variables, we use the **mean** for the line, rather than the median, and it explains why some of the ribbons look weird.

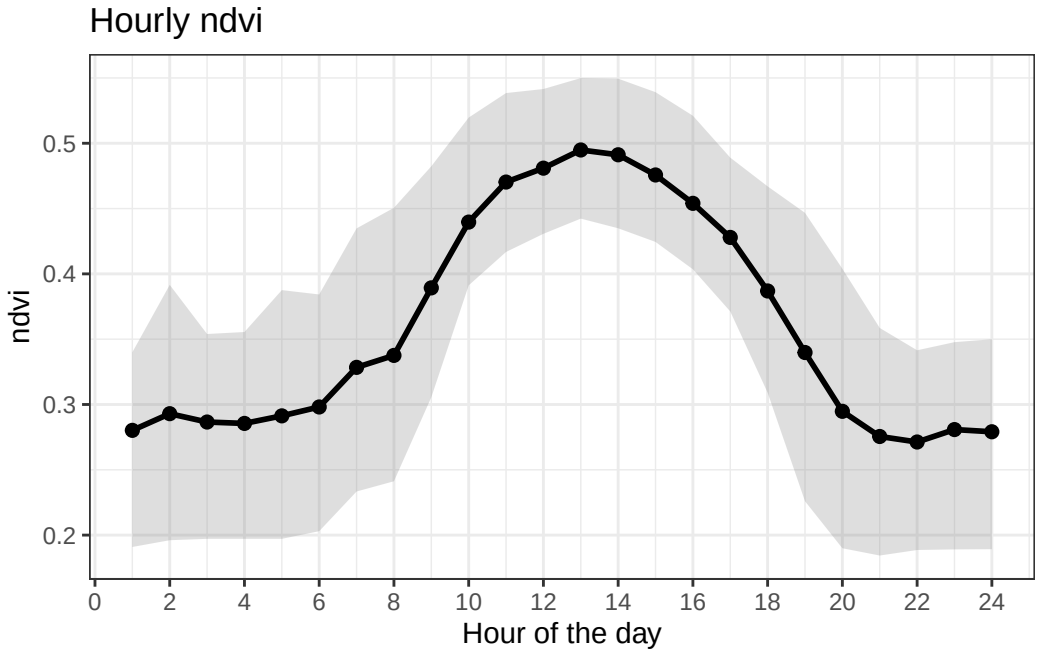
```r
# variables <- unique(buffalo_hourly_habitat_long$variable)

# variables to loop over
variables <- c("step_length", "ndvi", "herby", "canopy", "slope")

for(i in 1:length(variables)){

  # Filter data for the current variable
  var_data <- buffalo_hourly_habitat_long %>%
    filter(str_detect(variable, variables[i]))
```

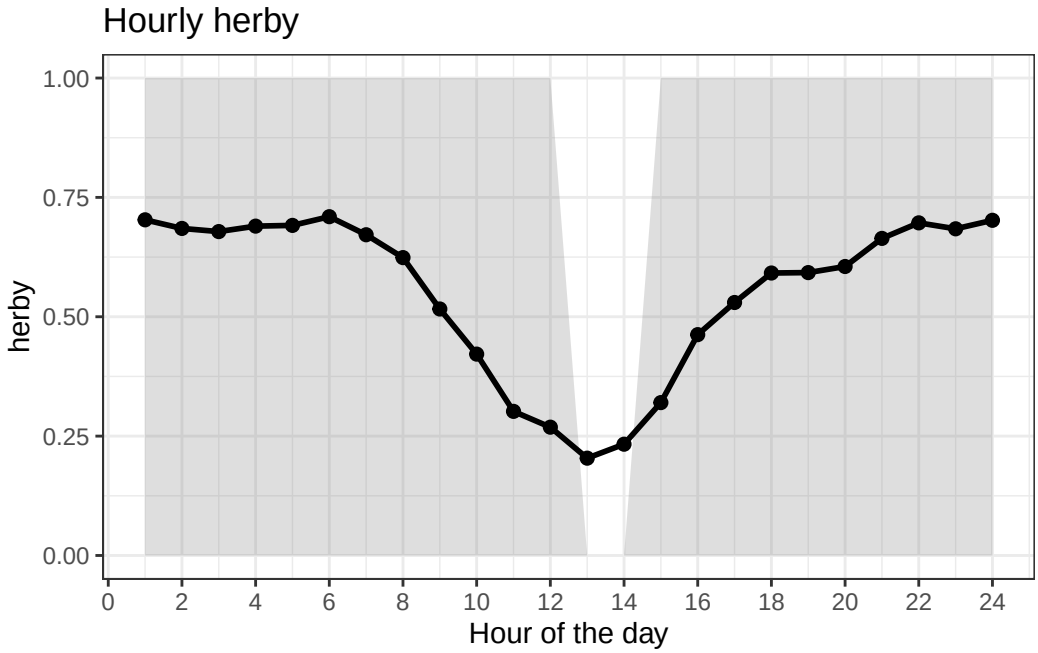
```
  # Extract specific quantiles for ribbons
  var_summary <- buffalo_hourly_habitat %>%
    select(hour_t2,
           mean = !!paste0(variables[i], "_mean"),
           q025 = !!paste0(variables[i], "_q025"),
           q25 = !!paste0(variables[i], "_q25"),
           median = !!paste0(variables[i], "_median"),
           q75 = !!paste0(variables[i], "_q75"),
           q975 = !!paste0(variables[i], "_q975"))

  plot <- ggplot(data = var_summary) +

    # ribbons
    # 95% ribbon
    # geom_ribbon(aes(x = hour_t2,
    #                 ymin = q025,
    #                 ymax = q975),
    #             alpha = ribbon_95_alpha, fill = "grey70") +

    # 50% ribbon
    geom_ribbon(aes(x = hour_t2,
                    ymin = q25,
                    ymax = q75),
                alpha = ribbon_50_alpha, fill = "grey50") +

    # lines
    geom_line(aes(x = hour_t2, y = mean), size = 1) +
    geom_point(aes(x = hour_t2, y = mean), size = 2) +

    scale_x_continuous(breaks = seq(0, 24, by = 2)) +
    labs(x = "Hour of the day",
         y = variables[i],
         title = paste0("Hourly ", variables[i])) +
    theme_bw()

  print(plot)

  # ggsave(paste0("outputs/exploratory_temporal_dynamics/buffalo_hourly_", variables[i], ".p
  #        plot = plot,
  #        width = 150, height = 100, units = "mm", dpi = 600)

}
```

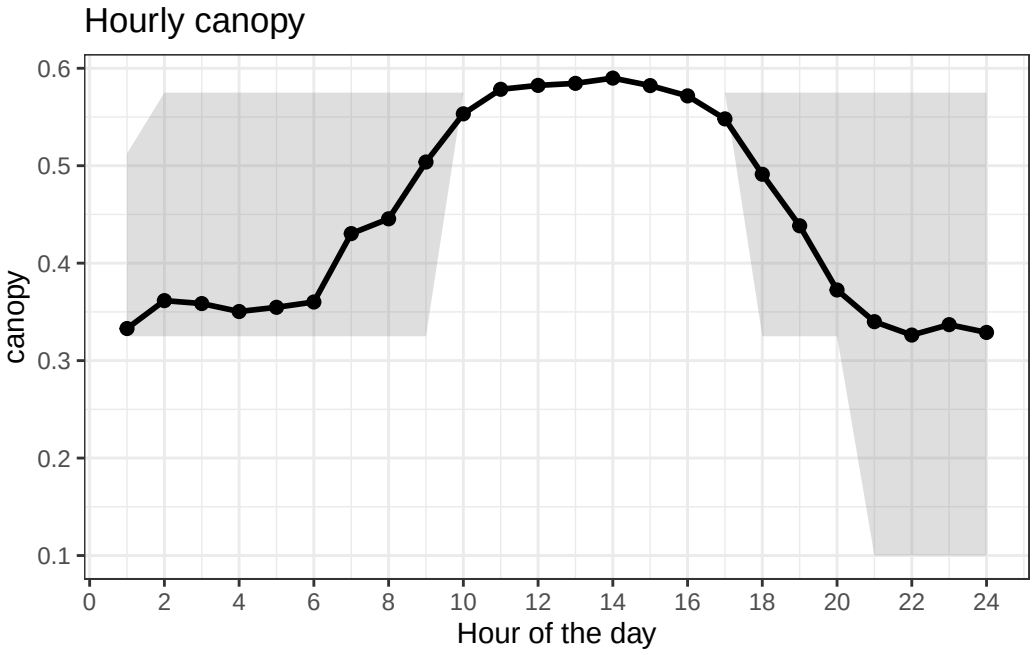
```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
```

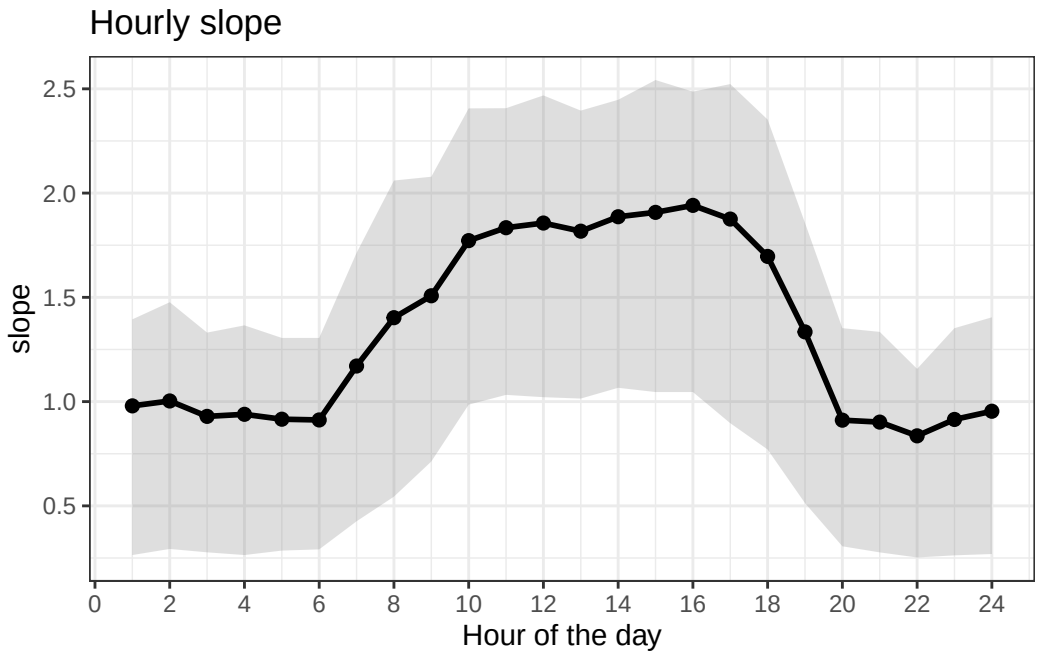
```
i Please use `linewidth` instead.
```






Hourly herby
herby
1.00
0.75
0.50
0.25
0.00
0
2
4
6
8
10
12
14
16
18
20
22
24
Hour of the day


Hourly canopy
canopy
0.6
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8
10
12
14
16
18
20
22
24
Hour of the day



References

Forrest, Scott W, Dan Pagendam, Michael Bode, Christopher Drovandi, Jonathan R Potts, Justin Perry, Eric Vanderduys, and Andrew J Hoskins. 2024. "Predicting Fine-scale Distributions and Emergent Spatiotemporal Patterns from Temporally Dynamic Step Selection Simulations." *Ecography*, December. https://doi.org/10.1111/ecog.07421.

Forrest, Scott W, Dan Pagendam, Conor Hassan, Jonathan R Potts, Christopher Drovandi, Michael Bode, and Andrew J Hoskins. 2025. "Predicting Animal Movement with deepSSF: A Deep Learning Step Selection Framework." *Methods in Ecology and Evolution*, August. https://doi.org/10.1111/2041-210X.70136.